

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name: Kraman Zhaniya Student ID: 230103077

Hardware / CPU: Apple Silicon (arm64), 8 threads Date & Section: 01.10.2026

Challenge 1: The Core Speedometer & Amdahl's Law

A. Baseline T_1 and maximum multi-thread runtime $T_{\max}$

T_1 (1 thread) = 0.3768 s;

$T_{\max}$ (8 threads) = 0.0766 s;

speedup = 4.92x (efficiency 61.5%).

Threads	Time (s)	Speedup	Efficiency %
1	0.3768	1.00	100.0
2	0.1914	1.97	98.4
4	0.0983	3.83	95.8
8	0.0766	4.92	61.5

Values are also
entered in Table 1.

B. Why is the warm-up call strictly required?

Numba is a JIT compiler: the first call to a decorated function triggers type inference, LLVM compilation to machine code and, with `parallel=True`, creation of the parallel runtime and thread pool. This costs hundreds of milliseconds to seconds. Without the warm-up call on 10,000 samples that one-time cost would be included in the first timed run (1 thread), inflating T_1 and therefore every speedup computed from it. The warm-up compiles the function and starts the thread pool so only the steady-state computation is measured.

C. Did efficiency stay at 100%? What explains the degradation?

No. Efficiency was 98.4% at 2 threads and 95.8% at 4 threads (near-linear), but dropped to 61.5% at 8 threads (speedup 4.92x instead of 8x). Contributing factors:

- Amdahl's Law: the serial portion (thread launch, scheduling, final reduction, call overhead) does not shrink; $\text{speedup} \leq 1 / (s + (1 - s)/N)$.
- SMT / Hyper-Threading: logical threads beyond the physical core count share the same execution units and caches, so each extra SMT thread adds only ~20–30% throughput rather than 100%.

- Heterogeneous cores (Apple M-series P/E cores, Intel P/E cores): efficiency cores are slower than performance cores.
- Frequency / thermal limits: with all cores loaded, turbo frequency drops and power or thermal throttling occurs.
- Shared resources (L3 cache, memory bus) — minor for Monte Carlo since there is almost no memory traffic, so efficiency stays high up to the physical-core count and drops beyond it.

D. Why does `inside_circle += 1` inside prange not corrupt the count?

Numba recognises this as a reduction pattern: a scalar initialised before the prange loop and updated with `+=` inside it. It gives every thread a private copy initialised to 0 (equivalent to OpenMP's `reduction(+:inside_circle)`); each thread accumulates independently and the partial sums are combined into the final value after the loop. There is no shared write, hence no data race and no need for locks or atomics.

Challenge 2: Load Imbalance & Dynamic Scheduling

```
Row-Parallel Render Time:    1.288 s
Column-Parallel Render Time: 1.247 s
Saved image: mandelbrot_output.png
```

A. `t_rows` vs `t_cols`

`t_rows = 1.288 s`; `t_cols = 1.247 s`. The two are nearly identical (columns about 3% faster) because the kernel is compute-bound: up to 1000 loop iterations per pixel versus a single store. In theory row decomposition has the more cache-friendly write pattern (contiguous memory), while column decomposition writes with a stride of $W \times 4$ bytes; however that cost is negligible next to the compute, so the 3% difference is within run-to-run noise and reflects mainly how the chunks happen to split the heavy central region of the fractal across threads.

B. Row-major layout and cache lines

NumPy arrays are C-contiguous: `img[r, 0..W-1]` are adjacent in memory, and a 64-byte cache line holds 16 int32 values. When threads own whole rows (`img[r, c]` with prange over `r`), each thread streams through its own contiguous memory, uses entire cache lines and never shares a line with another thread. When the parallel axis is the column (`img[r, c]` with prange over `c`), every write lands on a different cache line (stride $W \times 4$ bytes) and neighbouring elements of the same line are written by different threads. This causes false sharing: the line bounces between cores' caches under the MESI coherence protocol, raising cache misses and memory traffic.

C. Load imbalance analysis

Rows near the top/bottom of the image lie outside the set and escape after 2–3 iterations, while central rows ($cy \approx 0$) lie inside the set and run all $\text{max_iter} = 1000$ iterations — a difference of orders of magnitude. With static scheduling each thread gets one contiguous block of rows: the thread holding the top 20% finishes almost immediately and idles, while the thread holding the centre 20% is still computing. Total time equals the slowest thread, so utilisation is poor. With OpenMP dynamic scheduling (`schedule(dynamic, chunk)`) small chunks of rows are handed out on demand from a shared queue; a thread that finishes early simply takes the next chunk, so all threads stay busy until the work runs out, at the cost of a little scheduling overhead. (Numba's TBB/work-stealing backend balances load in a similar way.)

D. Exported image

Attached: `mandelbrot_output.png`

Challenge 3: Stencil Computation & Memory Bandwidth

```
Heat float64: 0.098 s | 6915.97 Megacells/s  
Heat float32: 0.064 s | 10477.26 Megacells/s | float64/float32 = 1.51x
```

A. Benchmark runtime and throughput

Runtime (float64) = 0.098 s; Throughput = 6915.97 Megacells/sec (also in Table 1).

B. float64 → float32

float32 runtime = 0.064 s (10,477.26 Megacells/sec), i.e. 1.51x faster than float64. A 1500×1500 float64 array is 18 MB, so the two arrays (36 MB) exceed typical L2/L3 caches and every time step streams from DRAM. float32 halves the bytes moved (and lets more of the grid fit in cache). The stencil does only ~6 FLOPs per cell against tens of bytes of traffic, so it is limited by memory bandwidth, not by arithmetic: halving bytes roughly halves the time, whereas compute capability is not the bottleneck. SIMD lanes also process twice as many float32 values per instruction.

C. Memory Roofline Model

Attainable performance = $\min(\text{peak compute}, \text{arithmetic intensity} \times \text{memory bandwidth})$. A stencil has very low arithmetic intensity (FLOPs per byte), so it lies under the sloped, bandwidth-limited part of the roofline. DRAM bandwidth is a single shared resource for the whole chip; one or two cores can already consume a large fraction of it. Once it is saturated, additional cores only wait on memory — their ALUs

sit idle — so doubling the core count does not double performance. The speedup curve flattens into a plateau (the memory wall), often after just 2–4 cores.

Student Result Scorecard (Table 1)

```

=== TABLE 1 ===
Benchmark      Threads  Time (s)  Speedup  Eff %
Monte Carlo 1 (base)  1      0.3768   1.00    100.0
Monte Carlo 2      2      0.1914   1.97    98.4
Monte Carlo 4      4      0.0983   3.83    95.8
Monte Carlo Max    8      0.0766   4.92    61.5
Mandelbrot (Rows)  8      1.288    N/A     N/A
Mandelbrot (Cols)  8      1.247    N/A     N/A
Heat Stencil (f64)  8      0.098    N/A     N/A    (6916.0 Mcells/s)

```

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	0.3768	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	0.1914	1.97x	98.4%
Challenge 1: Monte Carlo	4 Threads	120,000,000	0.0983	3.83x	95.8%
Challenge 1: Monte Carlo	Max Threads (8)	120,000,000	0.0766	4.92x	61.5%
Challenge 2: Mandelbrot (Rows)	Max Threads	2500 x 2500	1.288	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads	2500 x 2500	1.247	N/A	N/A
Challenge 3: Heat Stencil	Max Threads	1500 x 1500 x 300	0.098	N/A	N/A

Heat stencil throughput: 6915.97 Megacells/sec (float64); 10,477.26 Megacells/sec (float32, 1.51x faster).

Mandelbrot 2500x2500 (Render: 1.29s)

